Topic 5: FIR Filters

Summary based on J. H. McClellan, R. W. Schafer, and M. A. Yoder, Signal Processing First, Prentice Hall, 2003.

Topics: discrete-time systems, moving-average filters, general FIR filters, causality, unit impulses and impulse responses, convolution, block-diagram implementations, linearity, time invariance, and cascaded LTI systems.

> **Key idea.** An FIR filter forms each output sample as a finite weighted sum of present and delayed input samples. Its finite impulse response completely characterizes the system, and convolution provides the general rule for computing the response to any input.

5.1 Discrete-time systems

A discrete-time system is a computational process that transforms an input sequence into another sequence called the output signal.

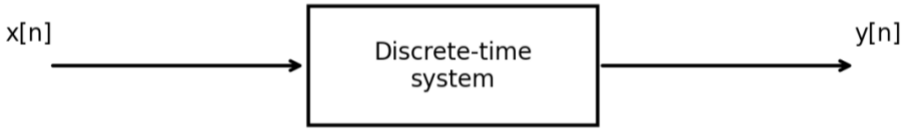


Figure 5.1. Block diagram of a discrete-time system.

In general, the operation of a system is denoted by

$$y[n] = \Gamma\{x[n]\}$$

A simple example is a system that scales the input by a constant:

$$y[n] = 2x[n]$$

5.2 Moving-average filter

A simple but useful transformation of a discrete-time signal is the moving average. Two or more consecutive samples are averaged to produce a new, smoother sequence. An FIR filter generalizes this idea. Averaging is useful when data fluctuate and should be smoothed before interpretation, for example in financial or measurement data.

Practical example. A long moving-average filter can reveal a slow trend hidden by rapid fluctuations. The textbook applies a 51-point causal averager to weekly stock-market data. Each output sample is the average of the current value and the previous 50 values:

$$y[n] = \frac{1}{51}\sum_{k=0}^{50} x\,[n-k]$$

The result is much smoother, but the causal filter also introduces a delay of approximately 25 samples. A centred moving average removes this visible delay when future samples are available.

Consider the three-point moving average of the finite triangular sequence shown in Figure 5.2.

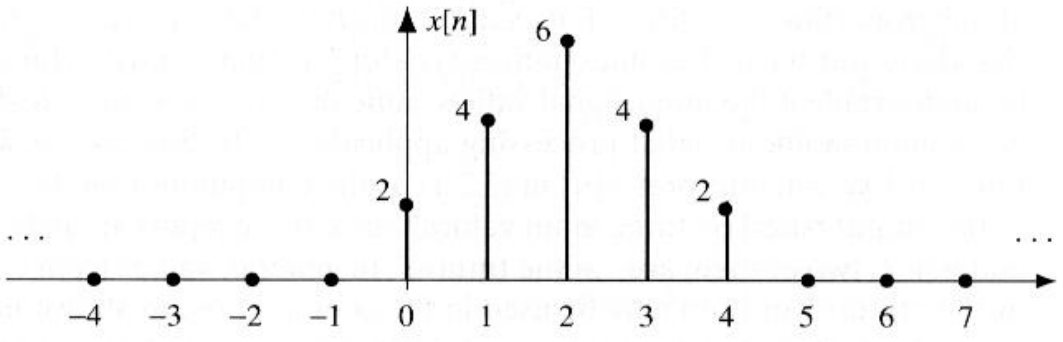


Figure 5.2. Finite-length input signal, x[n].

The output is calculated as

$$y[n] = \frac{1}{3}(x[n+1] + x[n] + x[n-1])$$

More generally, an L-point moving average can be written as

$$y[n] = \frac{1}{L}\sum_{k=0}^{L-1} x\,[n-k]$$

This expression is called a difference equation. It completely describes the FIR system because it allows the output to be calculated for every index n.

n	**n<−2**	**−2**	**−1**	**0**	**1**	**2**	**3**	**4**	**5**	**n>5**
x[n]	0	0	0	2	4	6	4	2	0	0
y[n]	0	2/3	2	4	14/3	4	2	2/3	0	0

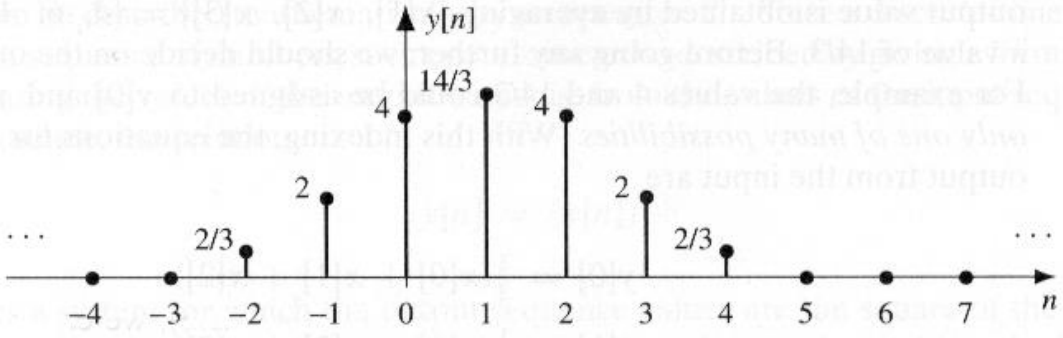


Figure 5.3. Output of the moving-average filter, y[n].

A filter that uses only present and past input values is causal: the output does not depend on data that have not yet arrived. A filter that uses future values is noncausal and cannot be implemented in real-time applications without delay.

A causal three-point moving average is

$$y[n] = \frac{1}{3}(x[n] + x[n-1] + x[n-2])$$

Worked example. For the finite sequence in Figure 5.2, evaluate the causal three-point average at n = 4. The filter uses the present sample and the two preceding samples:

$$y[4] = \frac{x[4] + x[3] + x[2]}{3} = \frac{2+4+6}{3} = 4$$

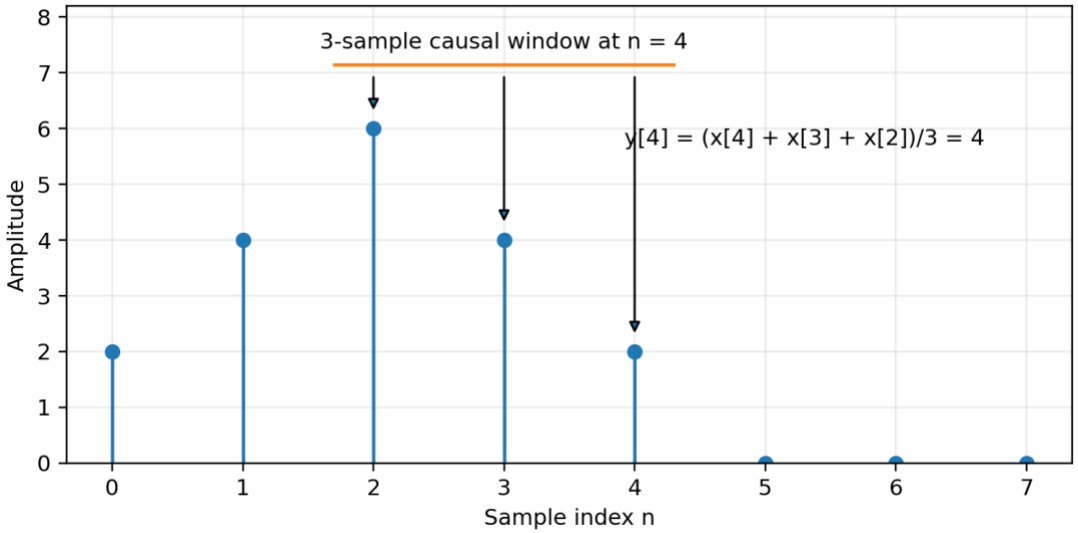


Figure 5.4. Sliding-window calculation for a causal three-point moving-average filter.

5.2.1 Causality and time alignment

The textbook first illustrates y[n] = (x[n] + x[n+1] + x[n+2])/3, a forward-looking average whose output starts before the nonzero input samples. Its causal counterpart is y[n] = (x[n] + x[n−1] + x[n−2])/3. Both average three numbers, but they differ by a two-sample time shift. Only the second expression can be computed at time n without future samples.

A centered average, (x[n−1] + x[n] + x[n+1])/3, has zero net shift for smoothly varying input away from boundaries but uses a future sample. One can implement it with a one-sample output delay if real-time latency is allowed. For odd-length symmetric coefficients of length L, a causal moving average introduces an effective delay of (L−1)/2 samples in its steady-state response.

For the textbook sequence x[0..4] = {2, 4, 6, 4, 2}, the forward-looking average gives y[−2..4] = {2/3, 2, 4, 14/3, 4, 2, 2/3}, while the causal average gives the same values indexed from n = 0 through 6. This indexing distinction is as important as the numerical result.

5.3 General FIR filter

The previous causal filter can be generalized to

$$y[n] = \sum_{k=0}^{M} b_k \, x[n-k]$$

The parameter M is the order of the FIR filter.

Worked example: coefficients and order. Let the FIR coefficients be b[0]=3, b[1]=−1, b[2]=2, and b[3]=1. The filter has order M=3 and length L=4, and its difference equation is

$$y[n] \;=\; 3x[n] \;-\; x[n-1] \;+\; 2x[n-2] \;+\; x[n-3]$$

This example makes clear that the coefficient list, the difference equation, and the impulse response are three equivalent descriptions of the same FIR filter.

$$L = M + 1$$

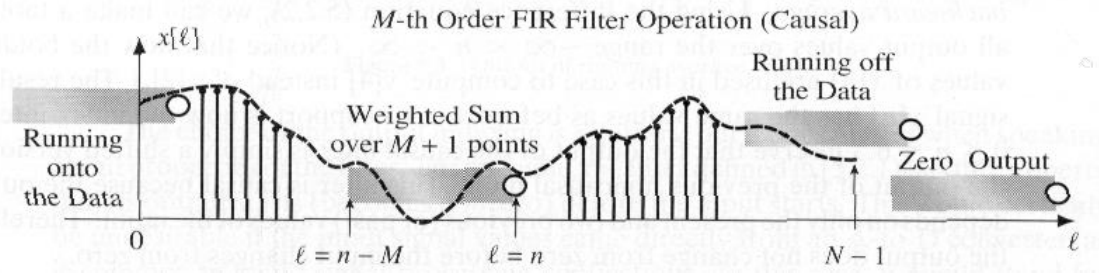


Figure 5.5. Operation of an order-M causal FIR filter as the processing window moves across the input.

5.3.1 Illustration of FIR filtering

Consider an input formed by a slowly increasing component plus a sinusoidal disturbance:

$$x[n] = (1.02)^n + \cos\left(\frac{2\pi n}{8} + \frac{\pi}{4}\right), \qquad 0 \le n \le 40$$

Treat the sinusoidal component as noise that should be reduced. For an order-2 causal moving-average filter,

$$y[n] = \frac{1}{3}\sum_{k=0}^{2} x\,[n-k]$$

The middle plot in Figure 5.6 shows that part of the sinusoidal fluctuation is removed, although the underlying trend is not recovered perfectly. Averaging over more samples gives stronger smoothing. With an order-6 filter,

$$y[n] = \frac{1}{7}\sum_{k=0}^{6} x\,[n-k]$$

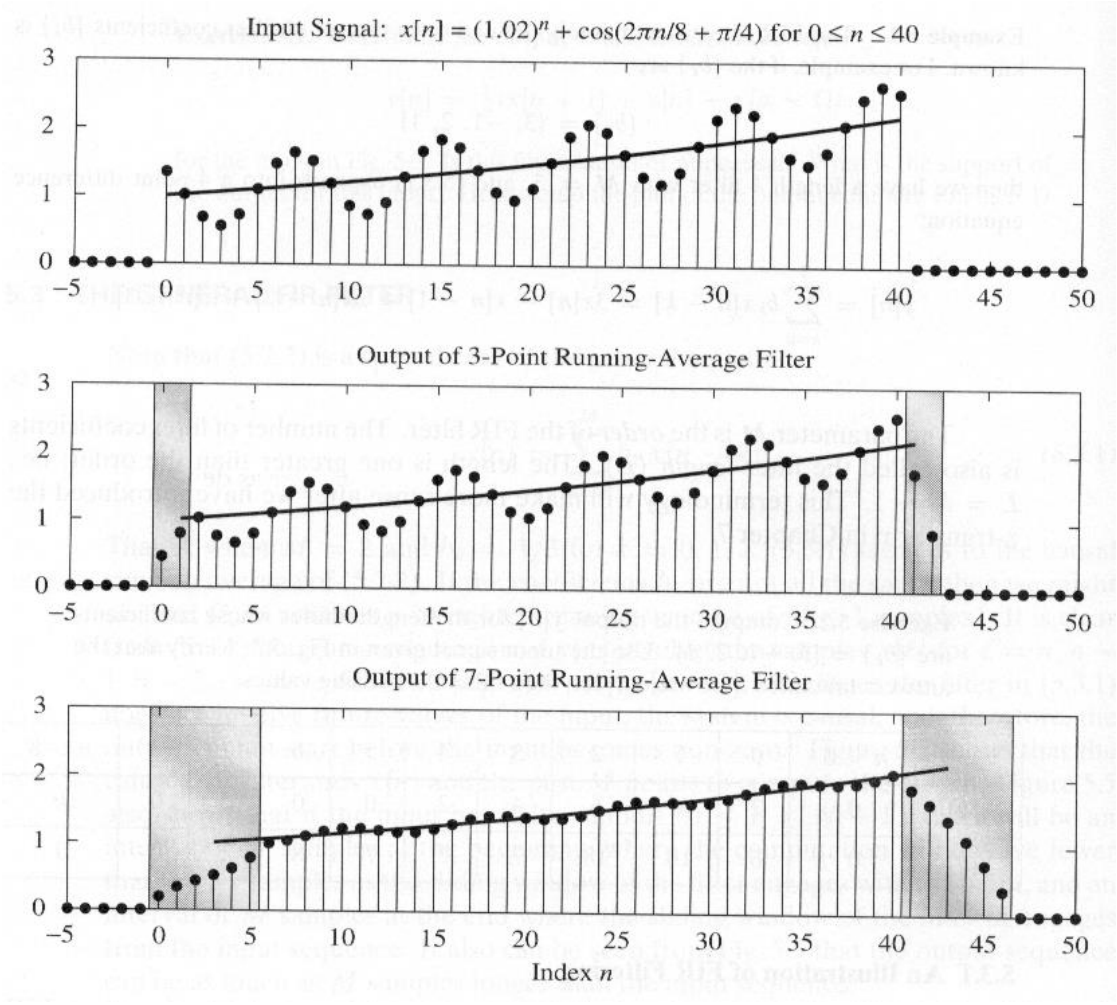


Figure 5.6. Moving-average filtering with 3-point and 7-point windows.

In Figure 5.6, the short average reduces the sinusoidal disturbance somewhat, while the seven-point average largely cancels the period-eight oscillation. Increasing the window length improves smoothing but increases the duration of onset and offset transients. A smooth output is not necessarily a more faithful output: rapid signal details may also be suppressed.

5.3.2 Unit impulse response

This section introduces three related ideas: the unit impulse sequence, the unit impulse response, and the convolution sum. The impulse response completely characterizes an FIR filter, and convolution provides a formula for calculating the output for any input when the impulse response is known.

5.3.2.1 Unit impulse sequence

The unit impulse is the simplest discrete-time sequence because it is nonzero only at n=0. It is denoted by the Kronecker delta:

$$\delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$$

A shifted impulse δ[n−3] is nonzero when n−3=0, that is, at n=3.

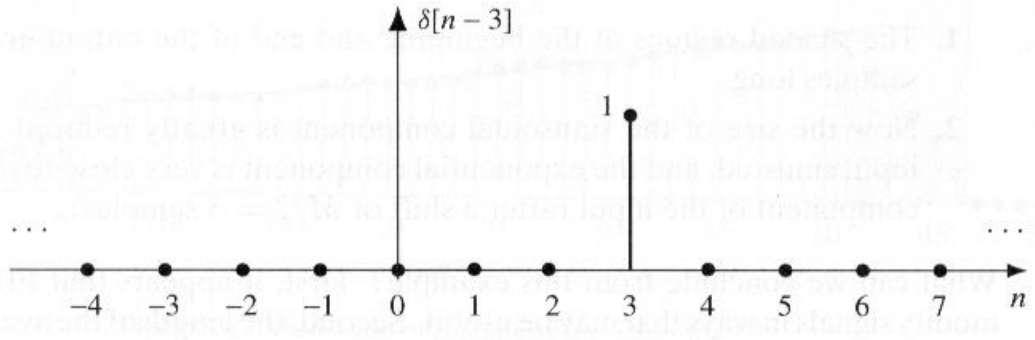


Figure 5.7. Shifted unit impulse sequence, δ[n−3].

Any sequence can be represented as a sum of scaled and shifted impulses. For the sequence in Figure 5.2,

$$x[n] = 2\delta[n] + 4\delta[n-1] + 6\delta[n-2] + 4\delta[n-3] + 2\delta[n-4]$$

In general,

$$x[n] = \sum_{k=-\infty}^{\infty} x\,[k]\delta[n-k]$$

5.3.2.2 Unit impulse response

When the input to an FIR filter is the unit impulse δ[n], the output is, by definition, the unit impulse response h[n], usually called simply the impulse response.



Figure 5.8. Block diagram defining the impulse response.

Substituting the unit impulse into the FIR difference equation gives

$$h[n] = \sum_{k=0}^{M} b_k\,\delta[n-k]$$

Only one term is nonzero for each value of n, so the impulse response is simply the sequence of filter coefficients:

$$h[n] = \begin{cases} b_n, & 0 \le n \le M \\ 0, & \text{otherwise} \end{cases}$$

Its duration is finite, which is why the system is called a finite impulse response filter.

Example: first-difference filter. The filter

$$y[n] = x[n] - x[n-1]$$
$$h[n] = \delta[n] - \delta[n-1]$$

responds to changes rather than constant levels. For a unit-step input u[n], all samples cancel except at the transition, so y[n]=δ[n]. This illustrates how a simple FIR filter can detect an onset or edge.

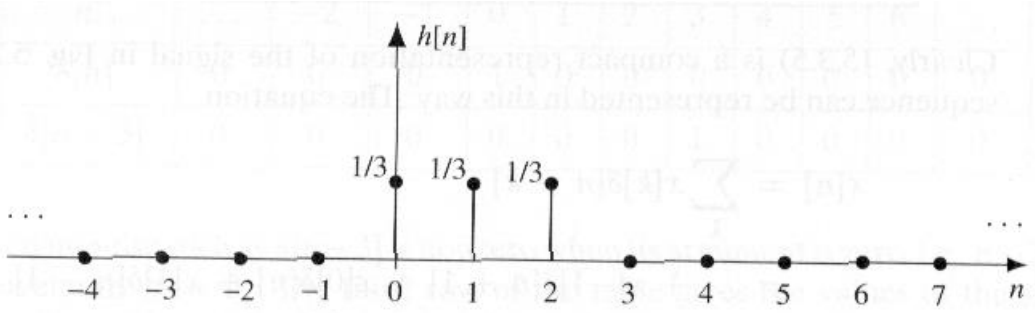


Figure 5.9. Impulse response of an order-2 moving-average filter.

5.3.2.3 Unit-delay system

An important elementary system delays, or shifts, the input by an integer number of samples:

$$y[n] = x[n - n_0]$$

For n_0=1 it is called a unit delay. A delay is the simplest FIR filter because all coefficients are zero except one. For a delay of three samples,

$$h[n] = \delta[n - 3]$$

5.3.3 Convolution and FIR filters

Because the FIR coefficients are identical to the impulse-response samples, the general FIR equation can be written as

$$y[n] = \sum_{k=0}^{M} h\,[k]x[n - k]$$

This is the finite convolution sum. The output is said to be obtained by convolving x[n] and h[n], written

Worked convolution example. For x[n]={2,4,6,4,2} and h[n]={3,−1,2,1}, the convolution is obtained by scaling and shifting the input once for each impulse-response sample and summing the aligned rows:

$$y[n] \;=\; x[n] \;*\; h[n]$$

The resulting output sequence is y[n]={6,10,18,16,18,12,8,2}. The output length is 5+4−1=8 samples.

	0	1	2	3	4	5	6	7
h[0]x[n−0]	6	12	18	12	6	0	0	0
h[1]x[n−1]	0	-2	-4	-6	-4	-2	0	0
h[2]x[n−2]	0	0	4	8	12	8	4	0
h[3]x[n−3]	0	0	0	2	4	6	4	2
y[n]	6	10	18	16	18	12	8	2

Figure 5.10. Numerical convolution as a sum of scaled and shifted input sequences.

5.3.4 Two views of numerical convolution

There are two equivalent ways to calculate y[n] = x[n] * h[n]. In the sliding-window view, fix the output index n, reverse and shift one sequence, multiply overlapping values, and sum. In the shifted-copy

view, take one delayed copy of x[n] for every coefficient h[k], scale it by h[k], then add all copies. The worked table in Figure 5.10 uses the shifted-copy view.

For the example x = {2,4,6,4,2} and h = {3,−1,2,1}, the value at n = 2 is y[2] = 3·6 − 1·4 + 2·2 = 18. A full linear convolution has 5 + 4 − 1 = 8 output values. The same computation can be checked with numpy.convolve(x, h, mode="full").

5.4 FIR-filter implementation

The FIR equation shows that implementation requires three operations: multiplying delayed input samples by the filter coefficients, adding the scaled values, and generating delays of the input sequence. Block diagrams provide a useful representation of these operations.

5.4.1 Basic blocks

The basic elements are a multiplier, an adder, and a unit-delay operator.

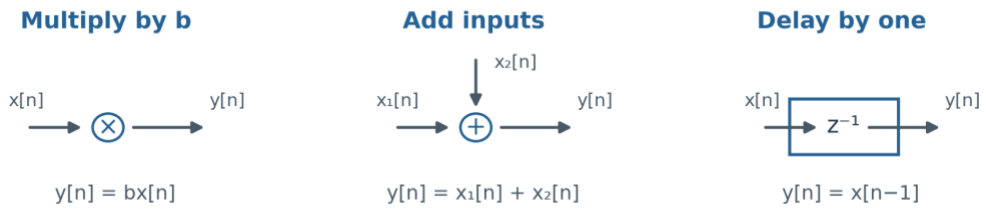


Figure 5.11. Basic blocks for constructing a discrete-time linear time-invariant system.

5.4.2 Block diagrams

Block-diagram notation specifies the interconnections among the basic elements and is especially useful for hardware structures. Figure 5.12 shows the direct-form realization of a third-order FIR filter.

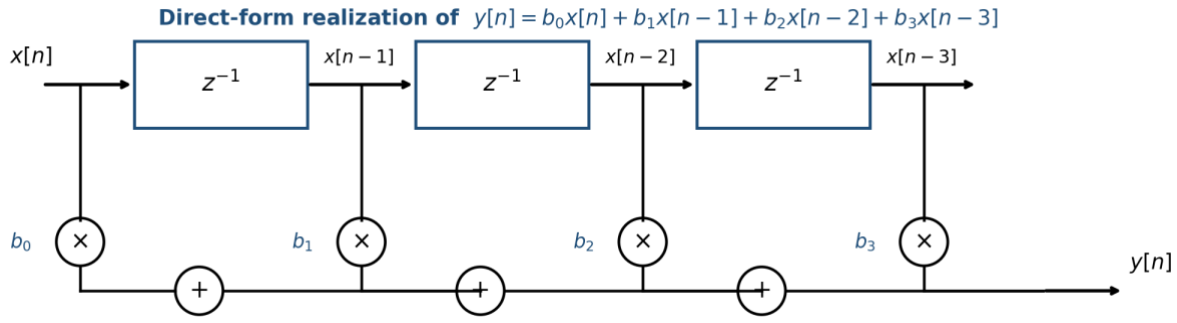


Figure 5.12. Direct-form block diagram of a third-order FIR filter.

The structure implements

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] + b_3x[n-3]$$

5.4.2.1 Alternative block diagrams

Many block diagrams can implement the same FIR filter, meaning that their external input-output behavior is identical. Figure 5.13 shows the transposed form of the same third-order filter.

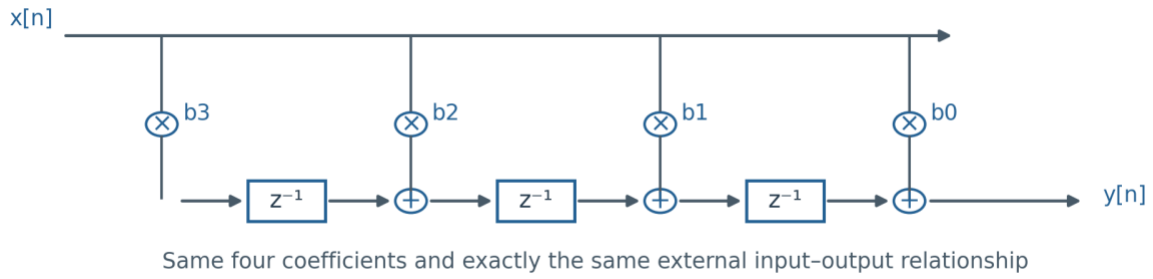


Figure 5.13. Transposed-form realization of the filter in Figure 5.12.

The direct-form FIR diagram stores delayed copies of the input and applies the coefficients to those samples. The transposed form stores partial accumulated sums instead. Both implement exactly the same difference equation and impulse response, despite having different internal signal values and computational organization.

For a length-L direct-form FIR, the straightforward realization has L multiplications, L−1 additions, and L−1 unit-delay elements per output sample. Equivalent realizations can be chosen for latency, parallelism, or numerical reasons. Changing the block diagram does not change the filter if its input–output equation remains the same.

5.5 Linear time-invariant systems (LTI)

Linearity and time invariance simplify mathematical analysis and provide a clearer understanding of system behavior.

5.5.1 Time invariance

A discrete-time system is time invariant if shifting the input by n_0 samples shifts the output by exactly the same amount. If

$$y[n] = \Gamma\{x[n]\}$$

then time invariance requires

$$\Gamma\{x[n - n_0]\} = y[n - n_0]$$

for every integer n_0.

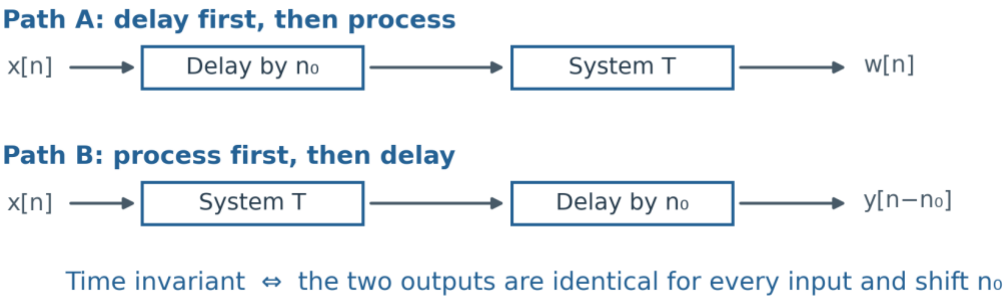


Figure 5.14. Test of the time-invariance property.

5.5.2 Linearity

A system is linear if, whenever

$$y_1[n] = \Gamma\{x_1[n]\}, \qquad y_2[n] = \Gamma\{x_2[n]\}$$

then, for any constants α and β,

$$\Gamma\{\alpha x_1[n] + \beta x_2[n]\} = \alpha y_1[n] + \beta y_2[n]$$

This is the superposition principle.

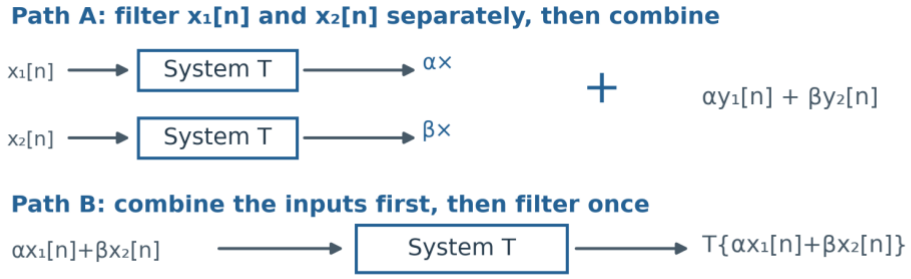


Figure 5.15. Test of the linearity property.

5.5.2.1 Distinguishing linearity from time invariance

The properties must be tested independently. For T{x[n]} = x[n]², shifting the input shifts the squared output by the same amount (time invariant), but T{2x} ≠ 2T{x} (not linear). For T{x[n]} = n x[n], superposition holds (linear), but shifting changes the factor n and the system is not time invariant. The time-reversal system T{x[n]} = x[−n] is linear but not time invariant.

The first-difference FIR, y[n] = x[n] − x[n−1], is both linear and time invariant. With unit-step input u[n], y[n] = δ[n]. It responds to the onset rather than to a sustained constant signal, offering a useful contrast with averaging.

5.5.3 The FIR case

The general FIR system is both time invariant and linear. For time invariance, delaying the input gives

$$\Gamma\{x[n - n_0]\} = \sum_{k=0}^{M} b_k\, x[n - n_0 - k]$$

whereas delaying the original output gives

$$y[n - n_0] = \sum_{k=0}^{M} b_k\, x[n - n_0 - k]$$

The expressions are identical. Linearity follows directly because the summation distributes over scaled sums:

$$\Gamma\{\alpha x_1[n] + \beta x_2[n]\} = \alpha\Gamma\{x_1[n]\} + \beta\Gamma\{x_2[n]\}$$

5.6 Convolution and LTI systems

The impulse response is a complete characterization of an LTI system.

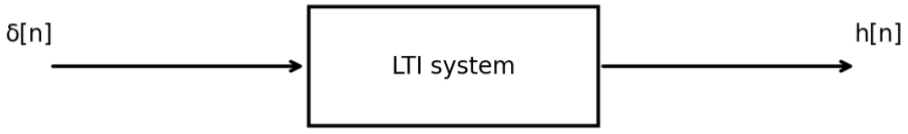


Figure 5.16. LTI system with an impulse input.

5.6.1 Derivation of the convolution sum

Write the input as a linear combination of scaled, shifted impulses:

$$x[n] = \sum_{k=-\infty}^{\infty} x\,[k]\delta[n-k]$$

By definition, the response to δ[n] is h[n]. Time invariance gives the response to a shifted impulse:

$$\Gamma\{\delta[n-k]\} = h[n-k]$$

Using linearity, the response to the complete input is

$$y[n] = \sum_{k=-\infty}^{\infty} x\,[k]h[n-k]$$

Therefore every LTI system can be represented by the convolution sum

$$y[n] = x[n] * h[n]$$

5.6.2 Properties of LTI systems

The algebraic properties of convolution correspond directly to properties of LTI systems.

5.6.2.1 Convolution as an operator

Convolution of x[n] and h[n] is denoted by *:

$$y[n] = x[n] * h[n]$$

Convolution with an impulse leaves a sequence unchanged:

$$x[n] * \delta[n] = x[n]$$

Convolution is commutative and associative:

$$x[n] * h[n] = h[n] * x[n]$$
$$(x[n] * h_1[n]) * h_2[n] = x[n] * (h_1[n] * h_2[n])$$

The general LTI convolution sum is y[n] = Σ_{k=−∞}^{∞} x[k]h[n−k]. If h[n] is zero outside 0 ≤ n ≤ M, this reduces to the finite FIR sum y[n] = Σ_{k=0}^{M} h[k]x[n−k]. Thus every fixed-coefficient FIR is LTI, but not every LTI system is FIR; some impulse responses have infinite duration.

5.7 Cascaded LTI systems

Two LTI systems connected in cascade can be implemented in either order because convolution is commutative. Their overall impulse response is

Worked cascade example. Suppose the first system has $h_1[n]=\{1,1,1,1\}$ and the second has $h_2[n]=\{1,1,1\}$. The equivalent impulse response is the convolution

$$h[n] = h_1[n] * h_2[n]$$
$$h[n] = \{1,2,3,3,2,1\}$$

The same result is obtained whichever system is placed first. A single FIR filter with these six coefficients therefore reproduces the complete cascade.

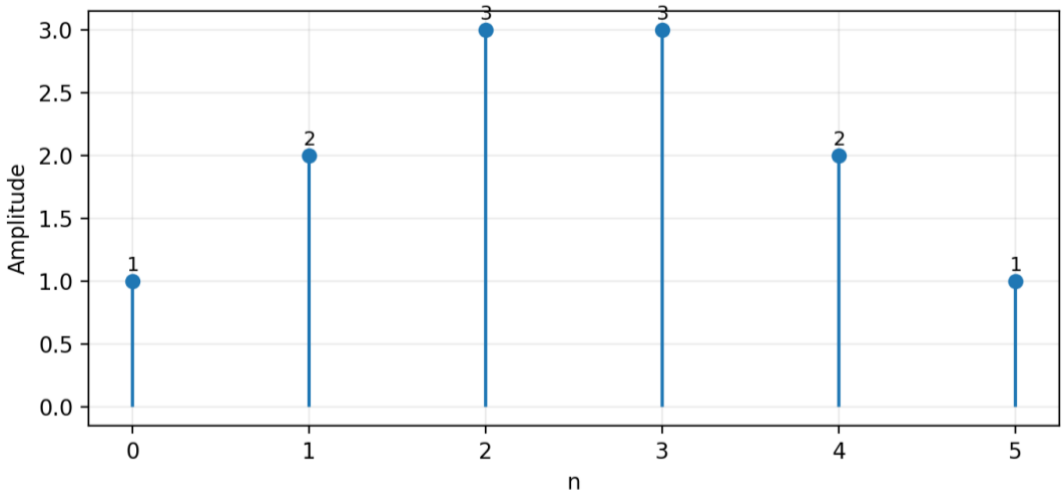


Figure 5.17. Equivalent impulse response of two cascaded moving-sum FIR filters.

$$h[n] = h_1[n] * h_2[n]$$

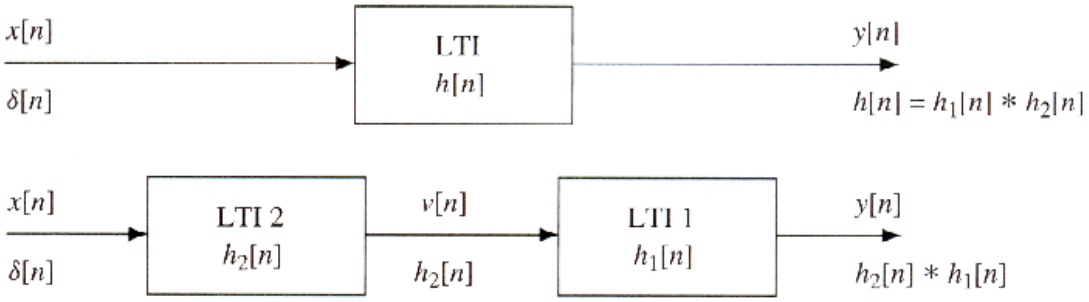


Figure 5.18. Two cascaded LTI systems and the equivalent system obtained by interchanging their order.

5.8 Notation

Symbol	Meaning
x[n]	Input discrete-time sequence
y[n]	Output discrete-time sequence
b_k	FIR filter coefficient at delay k
M	Filter order; the largest delay in an order-M FIR filter
L	Filter length, L = M + 1
h[n]	Impulse response of the system
δ[n]	Unit impulse (Kronecker delta)
u[n]	Unit-step sequence
n_0	Integer time shift or delay
*	Convolution operator
T{·}	System transformation operator
α, β	Arbitrary scalar constants used in the linearity test
$h_1[n]$, $h_2[n]$	Impulse responses of cascaded LTI systems

5.9 Practical examples and computational checks

This section consolidates the running-average, first-difference, convolution, and cascade ideas using the same numerical examples as the lecture slides.

5.9.1 The 3-point and 7-point averages

The example from Section 5.3.1 is $x[n] = (1.02)^n + 0.5\cos(2\pi n/8 + \pi/4)$ for $0 \le n \le 40$, and zero elsewhere. Comparing a causal 3-point average with a causal 7-point average illustrates suppression of rapid fluctuations and a longer startup transient. The graph below is recomputed from these values, rather than copied from the textbook.

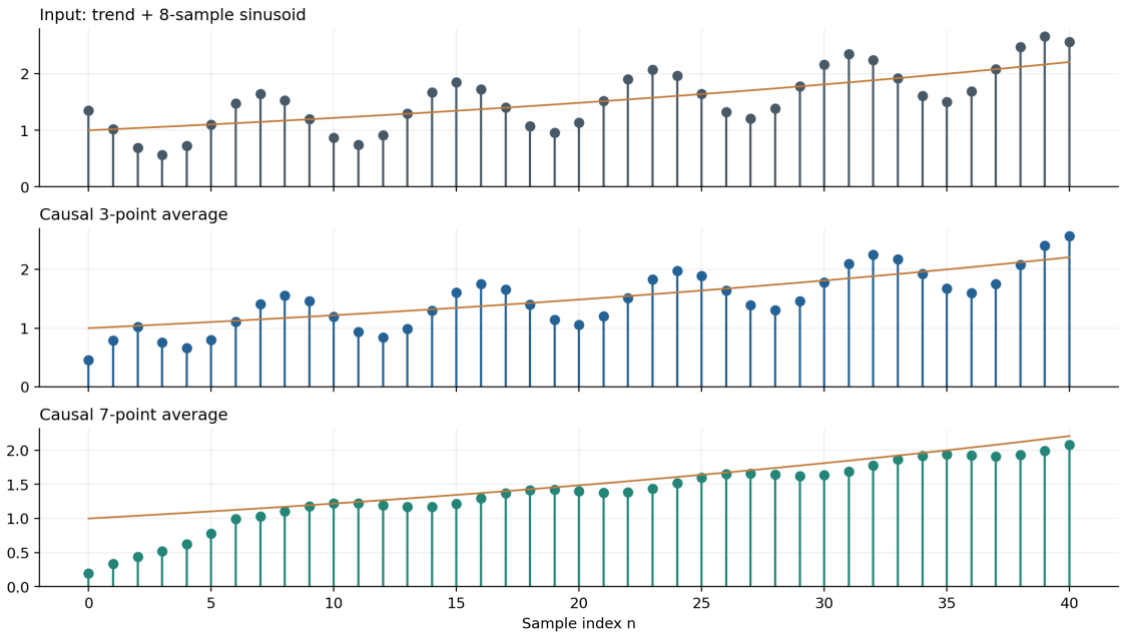


Figure 5.19. Recomputed textbook-style trend-plus-sinusoid input and causal averages (3 and 7 taps).

A longer averaging window is not automatically better: it suppresses more rapid variations but smears abrupt changes and creates more group delay. The central conceptual question is whether the eliminated variation is noise or useful signal.

5.9.2 The 51-point example and delay

The FIR chapter closes with a 51-point moving average applied to weekly stock-market data. A causal uniform 51-tap filter outputs the mean of x[n] and the previous 50 samples, with a 25-sample group delay. A centered version is possible offline, or by allowing an appropriate delay in real time. The following plot uses synthetic trend-plus-fluctuation data to demonstrate this time-alignment issue, not the historical stock series.

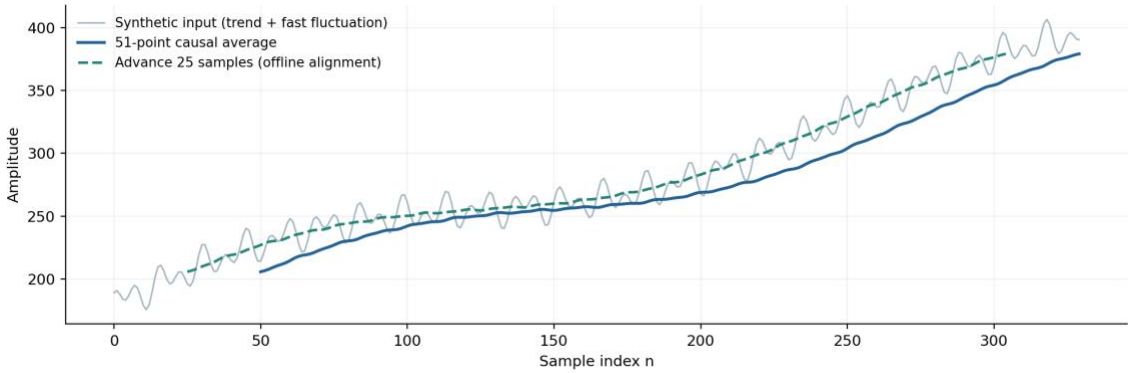


Figure 5.20. Illustrative 51-point causal running average and its 25-sample time alignment on synthetic data.

5.9.3 Computing and checking convolution

Consider x = [2,4,6,4,2] and h = [3,−1,2,1]. By the convolution sum, y[0]=6, y[1]=10, y[2]=18, and the full output is [6,10,18,16,18,12,8,2]. A short Python check is:

```
import numpy as np
x = np.array([2, 4, 6, 4, 2])
h = np.array([3, -1, 2, 1])
y = np.convolve(x, h, mode="full")
# [6, 10, 18, 16, 18, 12, 8, 2]
```

A useful exercise is to modify only h[1] and predict which output samples change before rerunning the calculation. This reinforces the interpretation of convolution as sums of scaled, shifted copies.

5.9.4 Cascading averaging filters

Two identical causal 3-point averaging filters have $h_1 = h_2 = [1/3,1/3,1/3]$. Their equivalent impulse response is h_eq = [1,2,3,2,1]/9. Unlike the single flat average, the cascade yields triangular tap weights. Since convolution is commutative, reversing the order of two LTI filters does not change the final output, though it may change intermediate signals.

5.10 What to remember and self-check

• A discrete-time system maps an input sequence x[n] to an output sequence y[n]. A filter is one such system; not every system is linear or time invariant.

• A causal FIR uses a finite number L=M+1 of present and delayed input samples. Its coefficients b[k] are exactly the samples of its impulse response h[k].

• A moving average performs smoothing, but the causal version also shifts/smears transients. The centered average uses future samples unless latency is introduced.

• Convolution can be calculated by a sliding weighted sum or by adding scaled, shifted copies. For a length-N input and length-L impulse response, the full output has N+L−1 samples.

• Linearity and time invariance are two independent properties. Together they imply that every LTI system is characterized by h[n] and the convolution sum.

• Direct-form and transposed-form block diagrams can implement the same FIR. Cascades of LTI systems have equivalent impulse response $h_1 * h_2$.

Quick self-check (answers follow): (1) Is $y[n]=x[n]^2$ linear? (2) Is $y[n]=nx[n]$ time invariant? (3) For $b=[2,-1,0,3]$, what are L and M? (4) For $y[n]=x[n]-x[n-1]$, what is $h[n]$? (5) Can a causal FIR use $x[n+1]$? (6) What is the impulse response of two cascaded 3-point averages?

Answers: (1) No. (2) No. (3) L=4 and M=3. (4) $\delta[n]-\delta[n-1]$. (5) No, not at time n without latency. (6) $[1,2,3,2,1]/9$.

References and course practice

McClellan, J. H., Schafer, R. W., and Yoder, M. A. (2003). Signal Processing First. Chapter 5, “FIR Filters”. Supplied chapter extract. DSPFirst, 2nd edition, Lecture 11 (“FIR Filtering Intro”) and Lecture 12 (“Linearity & Time-Invariance; Convolution”).

Related course materials: https://github.com/xserra/sis1-materials (class-slides, theory-summaries, examples, seminars, and labs). Example calculations and diagrams in this revision are adapted or recomputed; no claim is made that a particular notebook reproduces them.